

UNIVERSIDAD MINUTO DE DIOS
Corporación Universitaria Minuto de Dios

Especificación estructurada y Modelado UML del sistema de Gestión Parroquial

Johan Camilo Pedraos Yopasa

Docente: Cesar Alfonso Bolado Silva
Ingeniería de Sistemas

Corporación Universitaria Minuto de Dios

27/07/2026

Tabla de Contenido

Introducción	
Desarrollo	
1. Descripción del sistema	
2. Requisitos funcionales	
3. Requisitos no funcionales	
4. Estructura del modelo UML	
5. Trazabilidad requisitos – casos de prueba	
6. Puntos de control de calidad	
Planeación	
Conclusiones	
Referencias	

El presente trabajo tiene como objetivo construir un modelo de especificación estructurada del **Sistema de Gestión Parroquial (SGP)**, una aplicación de escritorio desarrollada con Python y el framework Flet, orientada a centralizar y digitalizar la administración de los sacramentos, eventos litúrgicos y la gestión administrativa de una parroquia o iglesia.

El modelado abarca la definición formal de requisitos funcionales y no funcionales, la elaboración de un **diagrama de casos de uso** que visibiliza la interacción de los actores del sistema con sus funcionalidades, y un **diagrama de clases conceptual** que refleja la estructura estática del dominio. A estos elementos se suman los criterios de aceptación por requisito, una matriz de trazabilidad básica entre requisitos y casos de prueba, y la identificación de los puntos de control de calidad a lo largo del ciclo de vida del software.

Desde la perspectiva del aseguramiento de la calidad, el modelo busca garantizar que cada funcionalidad tenga una definición verificable, un criterio medible de cumplimiento y al menos un caso de prueba asociado, siguiendo lineamientos del **Proceso Personal de Software (PSP)** y las prácticas de la ingeniería de software orientada a la calidad (Carmona-Murillo et al., 2019; García & López, 2017).

Descripción del sistema

Este aplicativo se desarrolló en base a tecnologías Python y el framework Flet, dando solución a las necesidades documentales de una parroquia católica. el aplicativo funciona de forma offline, sin dependencias de servidores externos ni conexión a internet, lo cual permite almacenar los datos registrados de forma local.

La base de datos en la cual se almacenan los registros es SQLite, la cual hace que sea más fácil la instalación y manejo en equipos de bajos recursos, este software permite gestionar los 5 sacramentos principales, (bautismo, primera comunión, confirmación, matrimonio y defunción), cuenta con funcionalidades para generar certificados en formato PDF conforme al Código de Derecho Canónico, y administra usuarios con control de acceso por roles.

Contexto organizacional.

Este sistema esta pensado para el personal administrativo o de apoyo de una parroquia, existe un equipo de desarrollo detrás de los nuevos requerimientos, solicitudes, control de llaves o licencia de software y mantenimiento al mismo. Sin área de QA independiente, ni

equipo de pentesting o seguridad informática.

Infraestructura híbrida: aplicativo local y base de datos local con una dependencia de servidor en nube para validar licencia o llave de producto.

2. Requisitos Funcionales

Se identificaron los siguientes requisitos funcionales a partir del análisis de cada módulo vista y controlador implementado en el desarrollo de esta aplicación.

ID	Nombre del Requisito	Descripción	Prioridad
RF01	Autenticación de usuarios	El sistema debe permitir el inicio de sesión con credenciales (usuario y contraseña). Contraseñas almacenadas con hash bcrypt.	Alta
RF02	Gestión de usuarios	El administrador puede crear, editar, activar/desactivar y consultar usuarios del sistema.	Alta
RF03	Control de permisos por rol	Cada usuario tiene permisos configurables por módulo (crear, editar, eliminar).	Alta
RF04	Registro de bautismos	Captura y almacena los datos del bautizado, padres, padrinos, sacerdote, fecha, libro y folio.	Alta
RF05	Registro de comuniones	Registra la primera comunión con datos del comulgante, padres, sacerdote y parroquia.	Alta
RF06	Registro de confirmaciones	Almacena información de la confirmación incluyendo padrinos y obispo confirmante.	Alta
RF07	Registro de matrimonios	Registra datos de ambos cónyuges, testigos, sacerdote y fecha del sacramento.	Alta
RF08	Registro de defunciones	Captura el acta de defunción con datos del difunto, familiares y sacerdote.	Alta
RF09	Gestión del calendario	Permite crear, editar y eliminar eventos litúrgicos con fechas de inicio y fin.	Media
RF10	Generación de reportes PDF	Exporta certificados sacramentales individuales con logo de la diócesis.	Alta
RF11	Búsqueda de registros	Filtros de búsqueda por nombre, fecha y municipio en cada módulo sacramental.	Media
RF12	Gestión de diócesis	Administra los datos institucionales de la diócesis: nombre, obispo, logo.	Media
RF13	Validación de licencia	El sistema consulta una API externa para validar la clave de licencia antes de permitir el uso.	Alta
RF14	Dashboard de estadísticas	Muestra indicadores consolidados de registros por módulo en la pantalla principal.	Media
RF15	Configuración del sistema	Permite ajustar parámetros como nombre de la parroquia, logo y ruta de exportación PDF.	Baja

3. Requisitos No Funcionales.

Los requisitos no funcionales muestran las características de calidad que debe cumplir el sistema independientemente de las funcionalidades específicas.

ID	Categoría	Descripción	Métrica
RNF01	Rendimiento	Las operaciones de consulta sobre la base de datos deben completarse en menos de 2 segundos para hasta 10 000 registros.	< 2 s / 10 000 reg.
RNF02	Seguridad	Las contraseñas deben almacenarse con hash bcrypt. Las sesiones expiran tras 30 min de inactividad.	Hash SHA256 + bcrypt
RNF03	Usabilidad	Un usuario no técnico debe poder registrar un sacramento en menos de 5 minutos con mínima asistencia.	< 5 min / tarea
RNF04	Disponibilidad	La aplicación debe funcionar en modo offline; sólo la validación de licencia requiere conexión.	99% offline
RNF05	Portabilidad	El ejecutable debe correr en Windows 10/11 sin instalación de dependencias adicionales.	Exe autocontenido
RNF06	Mantenibilidad	El código fuente debe seguir el patrón MVC con cobertura mínima de comentarios docstring del 60%.	60% docstring
RNF07	Integridad de datos	La base de datos debe implementar transacciones ACID con claves foráneas habilitadas.	PRAGMA FK=ON
RNF08	Escalabilidad	La arquitectura debe soportar la adición de nuevos módulos sacramentales sin rediseñar el núcleo.	Módulo en < 3 archivos

4. Diagrama de Casis de Uso.

Realizando el el diagrama de casos de uso se identificaron 3 actores principales,
Administrador: tiene el acceso total a todos los módulos y funcionalidades para administrar roles, permisos de usuario y licencia de producto.

Secretario-operador: es la persona responsable de realizar el registro diario de sacramentos, búsqueda y generación de reportes.

Sacerdote: Su rol se centra en la administración y consulta del calendario y el dashboard.

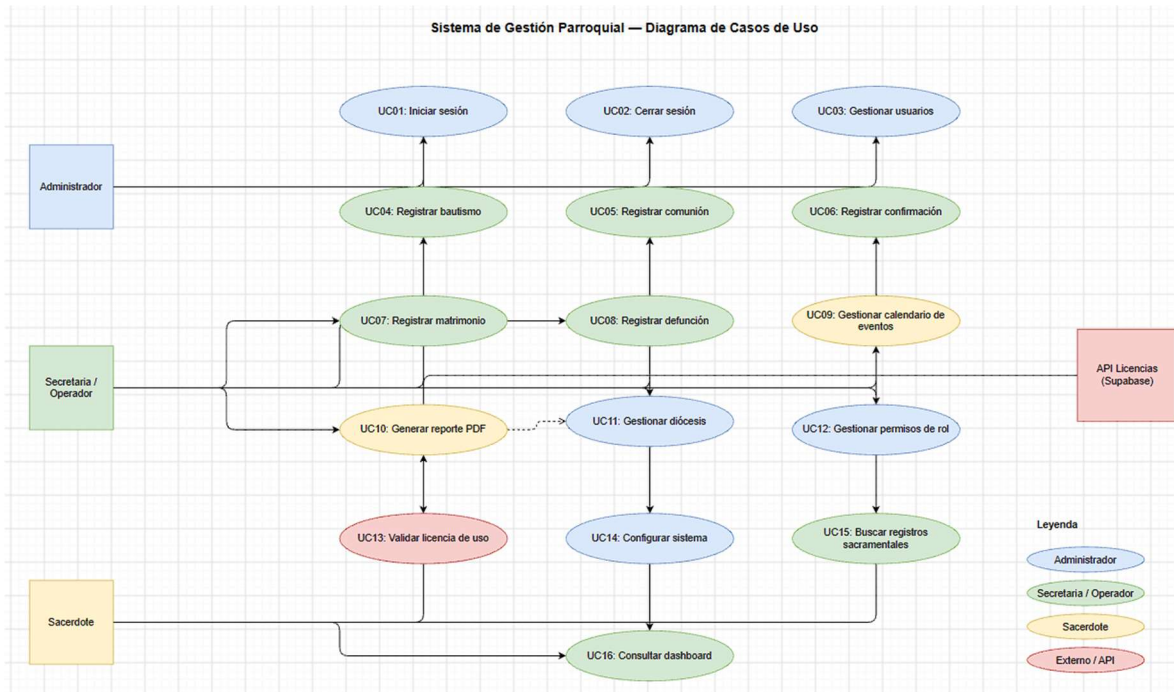
Actor Externo: admistrador de API Licencias (Supabase), administra consulta y renueva las licencias de uso individuales.

Se realizó el diagrama con 16 casos de uso agrupados en 3 categorías.

Gestión Administrativa: (UC01–UC03, UC11–UC15),

Registro Sacramental: (UC04–UC08)

Operación Litúrgica: (UC09–UC10, UC16).

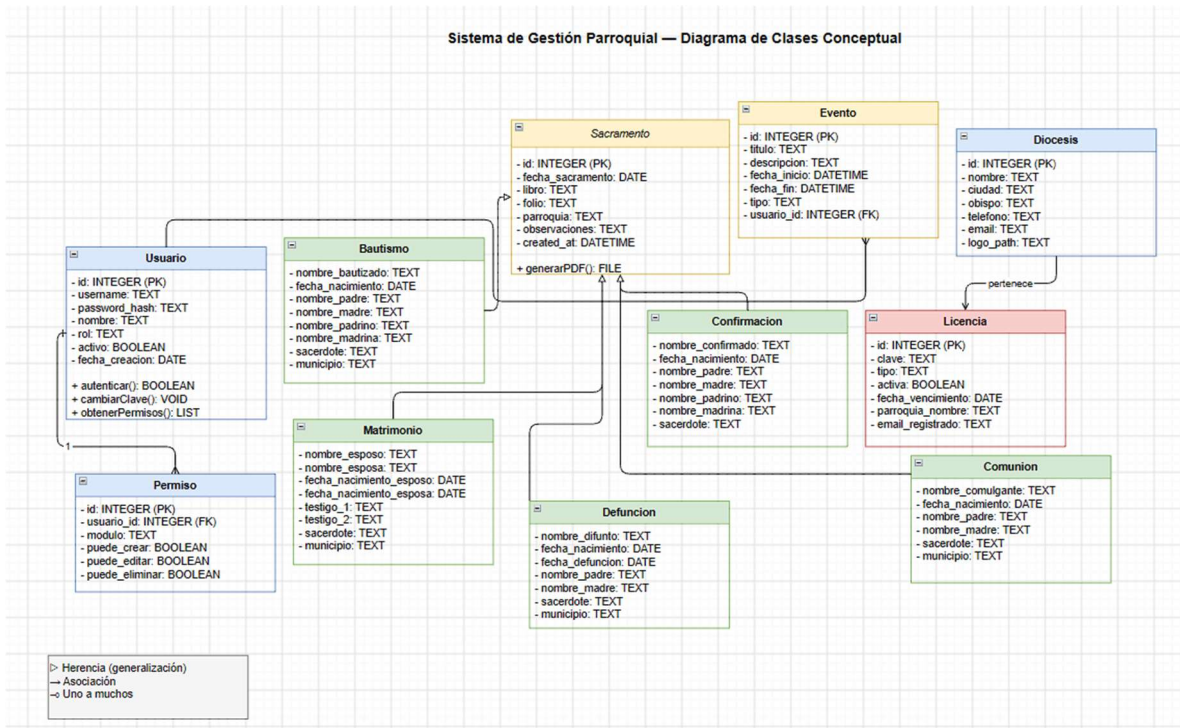


5. Diagrama de clases.

Este diagrama de clases refleja el dominio del negocio. La clase Sacramento centraliza los atributos comunes a todos los registros eclesiásticos (Libro, folio, fecha, sacerdote, parroquia), y cinco clases concretas la especializan mediante herencia:

Bautismo, Comunión, Confirmación, Matrimonio y Defunción, esta estructura reduce la duplicación de atributos y facilita la generación de reportes PDF.

La clase Usuario se relaciona en composición con Permiso (1.n), modelando el control granular de acceso por módulo. La clase Evento está asociada al usuario que lo crea. La clase Licencia se relaciona con Diócesis para identificar la instalación habilitada, y su validación se delega a la API externa.



6. Planeación.

Se realizó el análisis completo de la aplicación en 5 pasos

6.1. Exploración de código Fuente:

Se realiza análisis completo de la estructura MVC (Controladores, vistas, componentes y base de datos) las tablas en base de datos, la relación entre ellas realizada desde **DB Browser sqlite**.

6.2. Identificación de Actores y casos de uso.

En base a los módulos desarrollados y controles de acceso se definieron los 3 principales actores. (Administrador, Operador, Sacerdote). Se listaron los 16 casos a partir de las acciones disponibles en el sistema.

6.3. Elicitación de requisitos.

Los requisitos funcionales se extrajeron directamente del comportamiento y acciones de usuarios.

Los requisitos no funcionales se definieron mediante los criterios de calidad del modelo ISO/IEC 25010

6.4. Modelado UML en draw.io.

Se usó la herramienta draw.io para elaborar los diagramas de clases con jerarquía de herencia sacramental y asociaciones, y los diagramas de caso de uso con los actores involucrados.

Conclusiones

El ejercicio de modelado reveló que el sistema, aunque funcional en su implementación actual, carecía de una definición formal de sus requisitos funcionales y no funcionales desde las etapas tempranas del proyecto.

Omitir esta especificación desde el inicio representa un riesgo significativo para la calidad, ya que sin criterios medibles resulta imposible determinar objetivamente si el software alcanza sus objetivos o si los cumple solo de manera parcial.

Referencias.

Carmona-Murillo, J., Cortés-Polo, D., González-Sánchez, J. L., & Cárdenas, A. (2019). Modelo procedimental para la especificación de requisitos de calidad del software.

Revista Ibérica de Sistemas e Tecnologías de Informação, (E18), 329–342.
<https://www.proquest.com/scholarly-journals/modelo-procedimental-para-la-especificaci%C3%B3n-de/docview/2385371243>

García, A., & López, M. (2017). Aplicación del proceso personal de software en el aseguramiento de la calidad de proyectos de desarrollo de software.

Revista Ingeniería e Investigación, 37(2), 45–53.
<https://www.proquest.com/docview/2015771131>

Jiménez, R., Molina, F., & Torres, P. (2020). Aplicación del proceso personal de software en proyectos académicos de desarrollo de software.

Revista de Ingeniería de Software, 15(3), 112–128. <https://www.proquest.com/scholarly-journals/aplicaci%C3%B3n-del-proceso-personal-de-software-en/docview/2562270386>